ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

ФАКУЛТЕТ ПО КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ

КОМПЮТЪРНО И СОФТУЕРНО ИНЖЕНЕРСТВО

Нерелационни бази данни

Курсова работа

„Backlog refinement database“

Съставил: Ангел Любомиров Стойнов

Факултетен номер: 121222150

Група: 40

Съдържание

Въведение ... 3
Теоретична и технологична обосновка ... 4
Архитектура и реализация ... 5
Sessions ... 6
Stories ... 7
Votes ... 8
Задължителни CRUD операции ... 9
Създаване: ... 9
Актуализация: ... 9
Изтриване: ... 11
Четене: ... 11
Допълнителни заявки ... 14
Заявка, използваща агрегиране, групиране или сортиране: ... 14
Заявка, която комбинира данни от две различни колекции: ... 15
Контрол на достъпа до данните ... 16
Стъпка 5: ... 16
Стъпка 6: ... 17
Стъпка 7: ... 17
Цитирани източници ... 18
Използвани съкращения ... 18

Въведение

Backlog refinement срещата е ключова част от Agile церемониите [1]. В нея софтуерните разработчици, SCRUM master и Product Owner анализират, оценяват и разбиват задачите на по-малки подзадачи. Най-популярните начини за оценка са числата на Фибоначи или размери на дрехи – S, M, L и т.н. Всеки размер символизира сложността и очакваната продължителност на задачата. Ако задача е оценена с размер L или по-голям, тя се разбива на подзадачи [2].

Основният проблем, който се разглежда в курсовата работа, е липсата на напълно безплатно и централизирано решение за оценяване на задачи по време на refinement срещи. Съществуващите SaaS платформи често имат ограничения – лимитирани гласувания, ограничен брой потребители или необходимост от платен абонамент.

В резултат на това много екипи извършват оценяването ръчно – чрез чат приложения или устно, което води до:

- липса на автоматично изчисляване на крайната оценка;
- липса на централизирана история на гласуванията;
- възможност за човешки грешки при изчисленията;
- затруднено проследяване на предишни сесии.

За реализацията е избрана нерелационна база данни MongoDB [3]. Изборът е мотивиран от липсата на необходимост от сложни релационни връзки и от нуждата от гъвкава структура на данните. Оценките могат да бъдат представени като различни типове стойности – текстови или числови, което прави документо-ориентирания модел по-подходящ. Допълнително предимство е възможността за хоризонтално мащабиране и ефективна работа при голям брой гласове и заявки в реално време.

Теоретична и технологична обосновка

Нерелационните бази данни предлагат по-гъвкав модел на съхранение на информацията. Те не изискват фиксирана схема и позволяват динамично добавяне на нови полета към документите. Това ги прави подходящи за системи, при които структурата на данните може да се променя или разширява във времето.

В настоящия проект е използвана документо-ориентирана база данни MongoDB. Данните се съхраняват под формата на JSON документи (фиг. 1), което улеснява тяхната обработка и интеграция с уеб приложения. Всеки запис представлява самостоятелен документ, който може да съдържа вложени структури и различни типове данни. На фигура 1, полето finalSize може да бъде от няколко различни типа данни – int, double, string.

NoSQL модела елиминира необходимостта от сложни JOIN операции и намалява зависимостта между отделните структури [3]. Заявките са правени предимно през Mongosh.

```
{
  „_id": {
    „$oid": „6994c379616d90a4797c2908"
  },
  „sessionId": {
    „$oid": „6994c21b616d90a4797c2907"
  },
  „externalKey": „JIRA-101",
  „finalSize": „M",
  „revealed": true,
  „round": 2,
  „title": „Run LSR Lombok Recipes"
}
```

(Фиг. 1)

Архитектура и реализация

Данните са структурирани като JSON документи, което позволява лесно добавяне на нови полета при необходимост. Основните операции върху базата включват създаване на сесия, добавяне на задача, записване на глас, разкриване на резултатите и извличане на историята на гласуванията. MongoDB предоставя механизми за ефективно изпълнение на тези операции чрез стандартни заявки и агрегиращи механизми.

Тази архитектура осигурява добра мащабируемост, опростена структура на данните и висока производителност при работа в реално време.

Sessions

Sessions е колекция описваща конкретен refinement. Включва име, статус на срещата и нейните участници. Участниците са представени като част от масива participants. Те съдържат име на участника, професионално звание и иденфикационен номер (фиг. 2).


```
{
  "_id": {
    "$oid": "6994c21b616d90a4797c2907"
  },
  "name": "Spring refinement 1",
  "status": "IN_PROGRESS",
  "participants": [
    {
      "name": "Angel",
      "role": "DEV",
      "id": "p1"
    },
    {
      "name": "Simo",
      "role": "DEV",
      "id": "p2"
    }
  ]
}
```


(Фиг. 2)

Stories

Stories описва дадена задача, която предстои да бъде оценена. Съдържаща ключ към дадената сесия, външен ключ отговарящ на задачата в JIRA – за бърза проверка, размер на задачата, булева стойност revealed – дали крайният резултат от гласуването е визуализиран, броя рундове отнели за гласуване и заглавие на проблема (фиг. 3)


```
{
  „_id“: {
    „$oid“: „6999d3cfa33e156ef07c2910“
  },
  „sessionId“: {
    „$oid“: „6999d2f7a33e156ef07c2907“
  },
  „externalKey“: „JIRA-102“,
  „finalSize“: „L“,
  „revealed“: true,
  „round“: 1,
  „title“: „Implement OpenRewrite Security Recipe"
}
```


(Фиг. 3)

Votes

Votes представлява гласуването, което се извършва от всеки участник в сесията за всяка от избраните задачи. Votes съдържа ключове към задачата, брой рундове гласувания, ключ на гласуващия и сложността на задачата (фиг. 4).


```
{
  "_id": {
    "$oid": "6999d50ca33e156ef07c2919"
  },
  "storyId": {
    "$oid": "6999d3cfa33e156ef07c2910"
  },
  "round": 1,
  "participantId": "p1",
  "value": "L"
}
```


(Фиг. 4)

Задължителни CRUD операции

Създаване:

```
db.votes.insertOne({
  storyId: ObjectId("6999d3cfa33e156ef07c2914"),
  round: 1,
  participantId: "p5",
  value: "M"
})
```

(Фиг. 5 – създаване на вот)

Актуализация:

```
db.stories.updateOne(
  {
    sessionId: ObjectId("6994c21b616d90a4797c2907")
  },
  {
    $set: {
      externalKey: "JIRA-101",
      title: "Run LSR Lombok Recipes",
      round: 2,
      revealed: true,
      finalSize: "M"
    }
  }
)
```

(Фиг. 6 – актуализация чрез $set)

```
db.votes.updateOne(
  { _id: ObjectId(„6994cdad616d90a4797c2909“) },
  {
    $inc: {
      round: 1
    }
  }
)
```

(Фиг. 7 – актуализация чрез $inc)

```
      db.sessions.updateOne(
        { _id: ObjectId("6999d2f7a33e156ef07c290f") },
        {
          $push: {
            participants: {
              name: "Ivaylo",
              role: "SCRUM Master",
              id: "p5"
            }
          }
        }
      )
// резултат
participants: [
{ name: "Angel", role: "DEV", id: "p1" },
{ name: "Simo", role: "DEV", id: "p2" },
{ name: "Maria", role: "PO", id: "p3" },
{ name: "Nikolay", role: "DM", id: "p7" },
{ name: "Ivaylo", role: "SCRUM Master", id: "p5" }
]
```

(Фиг. 8 – актуализация чрез $push)

```
      db.sessions.updateOne(
        { _id: ObjectId("6999d2f7a33e156ef07c290f") },
        {
          $pull: {
            participants: { name: "Simo" }
          }
        }
      )
// резултат
participants: [
{ name: "Angel", role: "DEV", id: "p1" },
{ name: "Maria", role: "PO", id: "p3" },
{ name: "Nikolay", role: "DM", id: "p7" },
{ name: "Ivaylo", role: "SCRUM Master", id: "p5" }
]
```

(Фиг. 9 – актуализация чрез $pull)

Изтриване:

```
db.votes.deleteOne({_id: ObjectId('699f5632f52dc19e837c2907')})
```

(Фиг. 10 – изтриване по id)

Четене:

```
db.sessions.find({}, {status: "IN_PROGRESS", name: 1, _id: 0})

[

  { name: 'Spring refinement 1', status: 'IN_PROGRESS' },

  { name: 'Spring refinement 2', status: 'IN_PROGRESS' },

  { name: 'Spring refinement 3', status: 'IN_PROGRESS' },

  { name: 'Spring refinement 4', status: 'IN_PROGRESS' },

  { name: 'Spring refinement 5', status: 'IN_PROGRESS' },

  { name: 'Spring refinement 6', status: 'IN_PROGRESS' },

  { name: 'Spring refinement 7', status: 'IN_PROGRESS' },

  { name: 'Spring refinement 8', status: 'IN_PROGRESS' },

  { name: 'Spring refinement 9', status: 'IN_PROGRESS' },

  { name: 'Spring refinement 10', status: 'IN_PROGRESS' }

]
```

(Фиг. 11 – четене, като се визуализира само name, status)

```
db.sessions.find({"participants.name": "Angel"}, {participants:
{$elemMatch: {name: "Angel"}}, _id:0})

[

  { participants: [ { name: 'Angel', role: 'DEV', id: 'p1' } ] },

  { participants: [ { name: 'Angel', role: 'DEV', id: 'p1' } ] },

  { participants: [ { name: 'Angel', role: 'DEV', id: 'p1' } ] },

  { participants: [ { name: 'Angel', role: 'DEV', id: 'p1' } ] },

  { participants: [ { name: 'Angel', role: 'DEV', id: 'p1' } ] },

  { participants: [ { name: 'Angel', role: 'DEV', id: 'p1' } ] }

]
```

(Фиг. 12 – филтрира участници по име „Ангел“ и използва $elemMatch да извади само участници от масива с име „Ангел“)

```
db.votes.find({round: {$gt: 3}})
[
  {
    _id: ObjectId('6999d50ca33e156ef07c2939'),
    storyId: ObjectId('6999d3cfa33e156ef07c2918'),
    round: 4,
    participantId: 'p1',
    value: 'XXL'
  },
  {
    _id: ObjectId('6999d50ca33e156ef07c293a'),
    storyId: ObjectId('6999d3cfa33e156ef07c2918'),
    round: 4,
    participantId: 'p2',
    value: 'XL'
  },
  {
    _id: ObjectId('6999d50ca33e156ef07c293b'),
    storyId: ObjectId('6999d3cfa33e156ef07c2918'),
    round: 4,
    participantId: 'p3',
    value: 'XXL'
  },
  {
    _id: ObjectId('6999d50ca33e156ef07c293c'),
    storyId: ObjectId('6999d3cfa33e156ef07c2918'),
    round: 4,
    participantId: 'p7',
    value: 'XL'
  }
]
```

(Фиг. 13 – филтрира по рундове, като връща всички рундове, които са по-големи от 3)

```
db.votes.find({value: {$regex: "L"}}, {value: 1})
[
  { _id: ObjectId('6999d50ca33e156ef07c2919'), value: 'L' },
  { _id: ObjectId('6999d50ca33e156ef07c291b'), value: 'L' },
  { _id: ObjectId('6999d50ca33e156ef07c291c'), value: 'XL' },
  { _id: ObjectId('6999d50ca33e156ef07c2921'), value: 'XL' },
  { _id: ObjectId('6999d50ca33e156ef07c2922'), value: 'L' },
  { _id: ObjectId('6999d50ca33e156ef07c2923'), value: 'XL' },
  { _id: ObjectId('6999d50ca33e156ef07c2924'), value: 'L' },
  { _id: ObjectId('6999d50ca33e156ef07c2927'), value: 'L' },
  { _id: ObjectId('6999d50ca33e156ef07c292d'), value: 'L' },
  { _id: ObjectId('6999d50ca33e156ef07c292f'), value: 'L' },
  { _id: ObjectId('6999d50ca33e156ef07c2930'), value: 'XL' },
  { _id: ObjectId('6999d50ca33e156ef07c2933'), value: 'L' },
  { _id: ObjectId('6999d50ca33e156ef07c2939'), value: 'XXL' },
  { _id: ObjectId('6999d50ca33e156ef07c293a'), value: 'XL' },
  { _id: ObjectId('6999d50ca33e156ef07c293b'), value: 'XXL' },
  { _id: ObjectId('6999d50ca33e156ef07c293c'), value: 'XL' }
]
```

(Фиг. 14 – филтрира чрез regex, където се съдържа буквата „L“)

Допълнителни заявки

Заявка, използваща агрегиране, групиране или сортиране:

```
db.votes.aggregate([
  {
    $group: {
      _id: "$round",
      total: { $sum: 1 }
    }
  },
  {
    $sort: {
      _id: 1
    }
  }
])
[

  { _id: null, total: 1 },

  { _id: 1, total: 15 },

  { _id: 2, total: 13 },

  { _id: 3, total: 5 },

  { _id: 4, total: 4 }

]
```

(Фиг. 15 – групира по рундове, създава променлива total за общия брой, като сумира с 1. Сортира накрая по ASC)

Заявка, която комбинира данни от две различни колекции:

```
db.stories.aggregate([
   {
     $lookup: {
       from: "votes",
       localField: "_id",
       foreignField: "storyId",
       as: "votes"
     }
   },
   {
     $project: {
       _id: 0,
       title: 1,
       externalKey: 1,
       finalSize: 1,
       round: 1,
       votes: {
         round: 1,
         participantId: 1,
         value: 1
       }
       }
      }
])

  {
    externalKey: 'JIRA-101',
    finalSize: 'M',
    round: 2,
    title: 'Run LSR Lombok Recipes',
    votes: [ { round: 3, participantId: 'p1', value: 'M' } ]
  },
  {
    externalKey: 'JIRA-102',
    finalSize: 'L',
    round: 1,
    title: 'Implement OpenRewrite Security Recipe',
    votes: [
      { round: 1, participantId: 'p1', value: 'L' },
      { round: 1, participantId: 'p2', value: 'M' },
      { round: 1, participantId: 'p3', value: 'L' },
      { round: 1, participantId: 'p4', value: 'XL' }
    ]
  },
```

(Фиг. 16 – комбинира 2 колекции и извежда общ резултат)

Контрол на достъпа до данните

1. Редактиране на *C:\Program Files\MongoDB\Server\8.2\bin\mongod.cfg*
2. Маха се коментара на *Security*
3. Под *Security* добавяме: *authorization: enabled*
4. Рестарт на MongoDb
5. Създава се програмно роли
6. Създава се програмно потребители
7. Логин

Стъпка 5:

```
db.createRole({
  role: "reader",
  privileges: [
    {
      resource: {
        db: "refinementDb",
        collection: "stories"
      },
      actions: ["find"]
    },
    {
      resource: {
        db: "refinementDb",
        collection: "votes"
      },
      actions: ["find"]
    }
  ],
  roles: []
```

(Фиг. 17 – създаване на роля само за четене. Същото се прави и за останалите роли, разликата е в правата.)

Стъпка 6:

```
db.createUser({
  user: "DB_ADMIN",
  pwd: "Abcd1234",
  roles: [
    { role: "reader", db: "refinementDb" }
  ]
})
```

(Фиг. 17 – създаване на потребител с права само за четене)

```
db.createUser({
  user: "XSADMIN",
  pwd: "Abcd1234",
  roles: [
    { role: "writer", db: "refinementDb" }
  ]
})
```

(Фиг. 18 – създаване на потребител с права само за писане)

Стъпка 7:

```
mongosh -u DB_ADMIN -p Abcd1234 --authenticationDatabase refinementDb

db.votes.insertOne({ value: "L" })

MongoServerError[Unauthorized]: not authorized on refinementDb to execute
command […]
```

Цитирани източници

[1] D. Radigan, „Agile ceremonies,“ [Онлайн]. Available: https://www.atlassian.com/agile/scrum/ceremonies.

[2] M. Rehkopf, „Backlog refinement,“ [Онлайн]. Available: https://www.atlassian.com/agile/scrum/backlog-refinement.

[3] „MongoDb documentation,“ [Онлайн]. Available: https://www.mongodb.com/docs/.

[4] „Websockets API,“ [Онлайн]. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API.

[5] A. Amazon, „SQL,“ [Онлайн]. Available: https://aws.amazon.com/what-is/sql/.

[6] A. Amazon, „Object relational mapping,“ [Онлайн]. Available: https://aws.amazon.com/what-is/object-relational-mapping/.

Използвани съкращения

SaaS – software as a service

SQL – structured query language

ORM – object relational mapping

JSON – JavaScript object notation

ASC - ascending